

Investigating Techniques of Monte Carlo Integration

Charles Giulian

I. Background

II. Uniform Monte Carlo Integration

```
# Setup ggplot
library(ggplot2)
theme_update(plot.title = element_text(hjust = 0.5))
theme_update(plot.subtitle = element_text(hjust = 0.5))

# Compute estimation
f <- function(x) x^2
a <- 0
b <- 10
N <- 50000
x <- seq(a, b, length.out = N)
y <- f(x)

x_min = min(x)
y_min = min(y)
x_max = max(x)
y_max = max(y)

in_points <- c()
out_points <- c()

for (i in 1:N) {
  rand_point = c(runif(1, min = x_min, max = x_max), runif(1, min = y_min, max = y_max))
  if (rand_point[2] < f(rand_point[1])) {
    in_points <- rbind(in_points, rand_point)
  } else {
    out_points <- rbind(out_points, rand_point)
  }
}

V = (y_max - y_min) * (x_max - x_min)
I = length(in_points) / N * V

# Plot results
function_points <- data.frame(x = x, y = y)
in_data <- data.frame(x = in_points[, 1], y = in_points[, 2])
out_data <- data.frame(x = out_points[, 1], y = out_points[, 2])
```

```
ggplot(function_points) +
  geom_vline(aes(xintercept = x_min), color = "blue", linetype = "dashed") +
  geom_hline(aes(yintercept = y_min), color = "blue", linetype = "dashed") +
  geom_vline(aes(xintercept = x_max), color = "red", linetype = "dashed") +
  geom_hline(aes(yintercept = y_max), color = "red", linetype = "dashed") +
  geom_point(data = in_data, aes(x = x, y = y), size = 0.5, color = "green") +
  geom_point(data = out_data, aes(x = x, y = y), size = 0.5, color = "blue") +
  geom_line(aes(x = x, y = y), linewidth = 1.0, color = "black") +
  ggtitle("Uniform Monte Carlo Estimation for  $x^2$  on  $[0,10]$ ", subtitle = "n = 50000")
```

